# FemboyScript

FemboyScript — детерминированный слой игровой логики и состояний движка. У него две поверхности авторинга и два исполняющих backend:

- визуальные редакторы LogicGraph и StateTree в редакторе;
- компактный регистровый байткод, который исполняет VM редактора;
- cook-time C++26 AOT-backend, который собирает нативный плагин для билда.

Документ описывает поведение 0.4.0-alpha.1. FemboyScript 2.0 — это версия байткода и графового формата; версия движка — 0.4.0-alpha.1.

## Что есть в 0.4.0-alpha.1

| Слой | Статус |
|---|---|
| Визуальный LogicGraph | Панель редактора и граф в памяти |
| Визуальный StateTree | Канвас редактора и TOML-сохранение в сцене |
| Компилятор байткода | `CompileLogicGraph()` даёт канонический байткод |
| VM редактора | `ScriptVm::Run()` без аллокаций |
| AOT-транспайлер | Байткод в C++26, затем нативный плагин |
| Загрузчик AOT | `AotModule` проверяет ABI и LayoutHash компонентов |
| Отдельный текстовый парсер `.fbs` | `CompileTextScript()` — текстовое подмножество |
| Массивы, строки, map, пользовательские функции, корутины | В 0.4.0-alpha.1 отсутствуют |

Визуальный граф — основной authoring-путь для дизайнеров. Текстовый front-end — опциональная power-поверхность для version control, CI, модов и плотной логики; он находится в движке (`Engine/Script/TextScript.hpp`) и в текстовом режиме `VisualScriptPanel`. Он понижается в тот же канонический байткод, что и LogicGraph; у VM и AOT нет отдельного runtime-пути.

```
event update {
    let half = 0.5;
    self.Transform2D.Rotation = self.Transform2D.Rotation + half;
    if (self.Transform2D.Rotation > 3.14) {
        self.Transform2D.Rotation = 0.0;
    }
    log_float(self.Transform2D.Rotation);
}
```

Поддерживается: блоки `start`/`update`/`fixed_update`, `let`, присваивание, `self.Component.Field`, `if`/`else`, `+ - * /`, сравнения, числовые литералы и native-вызовы (`log_float`, `log_vec2`, `random_range`, ...). Циклы, массивы, строки и пользовательские функции — следующая версия языка.

## Модель исполнения

На сущности висит `ScriptComponent` с handle программы и флагами событий. `FemboyScriptSystem` читает handle, один раз выполняет `Start`, затем каждый кадр `Update`. `FixedUpdate` доступен через ту же таблицу entry-точек.

VM — регистровая машина:

- 128 бестиповых 4-байтовых регистров;
- 8-байтовые инструкции;
- до 65 536 инструкций и 1 024 констант на программу;
- до 32 component bindings и 256 field bindings;
- 16 event-функций и 32 native-ссылки;
- бюджет 16 384 шага на один `Run()`;
- никаких heap-аллокаций на пути исполнения;
- любая ошибка возвращается как `VmFault`, не через исключение.

Регистры хранят float, int32, uint32 или биты entity/handle. Тип определяет opcode. Доступ к компоненту резолвит layout hash один раз при bind, затем выполняет sparse-set lookup и прибавление offset поля.

## Конвейер

```
LogicGraph или StateTree
        |
        v
CompileLogicGraph()
        |
        v
канонический байткод FemboyScript (.fembak payload)
        |
        +--> ParseProgram() -> BindProgram() -> ScriptVm::Run()
редактор/runtime
```

```
        |
        +--> TranspileProgramToCpp() -> C++26 -> нативный плагин  билд
```

`ParseProgram()` — граница доверия. Он проверяет magic, версию, размеры таблиц, каждый индекс регистра, каждую цель jump, каждый component/field binding, диапазон аргументов native и соответствие opcode/field. После true интерпретатор не перепроверяет hot path.

`BindProgram()` резолвит layout hash компонентов через `EcsBridge`. Программа, собранная под другой `LayoutHash`, отклоняется до первой записи в поле.

### Визуальный LogicGraph

`VisualScriptPanel` редактирует `LogicGraph`:

- entry-узлы `Event Start/Update/FixedUpdate`;
- математика (float, int, Vec2);
- get/set полей Mirror-компонентов;
- branch и сравнения;
- native-узлы (`LogFloat`, `LogInt`, `LogVec2`, `RandomRange`, `SpawnPrefab`, `DespawnSelf`, `PlaySound`, `GetDeltaTime`, `GetFrameIndex`);
- типизированные связи и запрет циклов.

Граф — fixed-capacity, trivially copyable payload и помещается в один CommandStack external buffer. Undo/redo редактора работает с графом как с данными, а не с C++-объектами.

Компиляция: `CompileLogicGraph()`. Компилятор отклоняет:

- отсутствие event entry;
- неподключённые обязательные пины;
- несовпадение типов;
- циклы;
- превышение таблиц field/component;
- превышение лимита инструкций или регистров.

### Визуальный StateTree

`AIStateTreeCanvasPanel` редактирует `StateTreeDefinition`. Runtime-модель плоская и change-driven, это не обход behaviour tree:

- фиксированная таблица до 64 узлов и 32 переходов;
- максимальная глубина 4;
- сущность хранит активный путь (`ActiveStates[4]`) и blackboard;
- evaluator — данные (`TagsAny`, `TargetWithinRadius`, `FloatBelow`,

`StateTimeAtLeast`, …);

- task выдаёт intent через `AICommandComponent` и command buffer;
- сущность в устойчивом состоянии делает один evaluator и один task.

Панель пишет скомпилированное дерево в TOML сцены:

```
[ai_state_tree]
version = 1
tree_id = 1
root = 0
node_count = 3
transition_count = 1

[[ai_state_tree.node]]
index = 0
kind = 0
parent = 65535
first_child = 1
next_sibling = 65535
child_count = 1
depth = 1
enabled = true
evaluator_kind = 0
evaluator_tag_mask = 0
evaluator_threshold = 0.0
evaluator_float_index = 0
task_kind = 0
task_tag_mask = 0
task_speed = 128.0
task_range = 32.0
task_flags = 0
position = [0.0, 0.0]
```

`kind`: `0` Root, `1` Selector, `2` Leaf. `parent`, `first_child`, `next_sibling` — индексы; `65535` означает «нет». `evaluator_kind` и `task_kind` используют порядок enum из `StateTree.hpp`. `position` — данные канваса, не runtime.

Кнопка `Save AI TOML` перезаписывает только блок `[ai_state_tree]`, сохраняя остальную сцену. `Load AI TOML` проверяет блок и перестраивает канвас. Ручное редактирование допустимо, но индексы и linked-list поля должны быть согласованы.

### Формат байткода

Заголовок — одна 64-байтовая cache line:

```
magic u32, version u16, flags u16,
codeCount u32, constantCount u32, fieldCount u32, componentCount u32,
```

```
functionCount u32, nativeCount u32, registerHighWater u32, reserved u32,
sourceHash u64, graphHash u64, reserved2 u64
```

Таблицы идут после заголовка: code, constants, fields, components, functions, native ids. Каждая таблица начинается с фиксированного выравнивания; общий размер выровнен по `max_align_t`. Формат канонический и детерминированный: один граф даёт побайтово одинаковый результат на любой машине.

Инструкция — `op:u8 dst:u8 src:u8 flags:u8 operand:u16 reserved:u16`.

Набор opcode закрыт: `Nop`, `Halt`, load/move, float/int арифметика, битовые int-операции, сравнения, jump, entity/field access, native call, spawn/despawn и random. Новая функция — это native, а не новый opcode.

### Коды ошибок VM

`InvalidProgram`, `MissingComponent`, `BadEntity`, `BadNative`, `BadField`, `BadRegister`, `StepLimit`, `DivisionByZero`, `NoRandomSource`, `NoCommandBuffer`, `SpawnUnavailable`, `UnknownOpcode`.

`ScriptRuntime::ReportFault()` пишет не более одного сообщения на слот программы. `Start` помечается запущенным даже после fault, чтобы сломанный Start не заливал лог на 60 Hz.

### AOT-бэкенд

`AotBuilder` работает на worker-потоке:

1. `TranspileLogicGraphToCpp()` компилирует граф в байткод, валидирует его и выдаёт плоскую C++26-функцию на событие;
2. сгенерированный код включает только `Engine/Script/AotAbi.hpp`;
3. доступ к компонентам идёт через `Aot::Context::Component(LayoutHash)`;
4. плагин экспортирует `fby_aot_module()` — `Module` с хешами компонентов и указателями `start/update/fixedUpdate`;
5. `AotModule::Load()` проверяет magic, ABI version, LayoutHash компонентов и наличие хотя бы одного события до исполнения.

В сгенерированном коде нет VM-dispatch и чтения байткода. Контракт LayoutHash тот же, что у редактора. Устаревший плагин отклоняется, а не пишет в изменившийся компонент.

Арифметическая совместимость обеспечена: integer add/sub/mul/neg используют определённое unsigned-переполнение; `INT_MIN / -1` и `INT_MIN % -1` защищены;

нефинитные float-константы генерируются как `std::numeric_limits`.

Совместимость AOT закрыта в текущем ABI (версия 7): host-callback лога знаковый (`logI32`), `DespawnSelf` возвращает тот же результат 1/0, который пишет VM, физические и камерные callback'и общие с VM, а сгенерированный код вызывает хук `fault(code, pc)`; телеметрия fault остаётся в `AotModule`, а не теряется молча.

### Рабочий процесс редактора

6. Откройте `VisualScript` и соберите LogicGraph либо `AI State Tree` и соберите дерево состояний.
7. Скомпилируйте граф и проверьте число узлов/инструкций/регистров.
8. Сохраните StateTree-блок в TOML сцены; LogicGraph в 0.4.0-alpha.1 — payload редактора.
9. Назначьте `ScriptComponent` handle программы на сущность.
10. Запустите PIE; fault появится в Console/LogBoy с программой, сущностью и pc.
11. Для билда включите AOT и запустите `fby_aotc` через release pipeline.

Итоговый плагин проверит `AotModule`.

### API физического бэкенда

Бэкенд по умолчанию — `Auto`. Три native доступны в любом скрипте и одинаково работают в VM и AOT:

```
backend = physics_get_backend()        // 1 Box2D, 2 DOD, -1 недоступно
caps = physics_get_capabilities()      // бит 0 соединения, 1 CCD, 2 soft, 3
детерминизм, 4 большие миры
ok = physics_set_backend(backend)      // обновляет политику для следующего
мира
```

`physics_set_backend` не переключает живой солвер. Он обновляет политику, которая попадёт в следующий Play-In-Editor или запуск игры; текущий мир продолжает работу, поэтому смена в середине кадра не может испортить handles тел. Проверяйте возможности до создания графа, зависящего от соединений или CCD.

В палитре LogicGraph те же операции — `Physics Get Backend`, `Physics Set Backend` и `Physics Capabilities`. Выходной пин — Float; Set принимает Exec и Float-селектор бэкенда. Старые графы без этих узлов мигрируют как `Auto`.

Снаряды настраиваются через отражённый компонент `Projectile2D`, а не через

отдельный native: `Mode` (0 Auto, 1 Hitscan, 2 Swept, 3 Dynamic), `Speed`, `Radius`, `MinWallThickness`, `Direction`, `MaxSubsteps` и `CollisionMask` задаются обычными узлами полей компонента.

### Движение камеры и искажения

API камеры работает с активной `CameraStackSystem` и не отдаёт сырые handles. Эффекты косметические и по умолчанию не входят в rollback-снапшоты.

```
camera_add_trauma(amount)
camera_distort(flags, strength, paramX, paramY, duration)
camera_offset(x, y, duration)
camera_zoom(zoom, duration)
camera_roll(radians, duration)
camera_clear_effects()
camera_set_motion_scale(scale)
```

Флаги `camera_distort`: `1` двойное зрение, `2` непропорциональное растяжение (`paramX/Y` — масштабы X/Y), `4` радиальное искажение, `8` хроматическое расщепление, `16` низкочастотное покачивание (`strength` — амплитуда, `paramX` — частота). Состояние опьянения или отравления игра определяет сама и переводит в эти универсальные примитивы; движок не хранит игровое состояние. В визуальной палитре есть `Camera Add Trauma`, `Camera Distort`, `Camera Offset`, `Camera Zoom`, `Camera Roll`, `Camera Clear Effects` и `Camera Motion Scale`. Они компилируются в те же natives, а AOT-плагины вызывают версионированные camera-callback'и.

### Лимиты и поведение при ошибках

- Больше 32 программ — `script program table is full`. Вызывайте `UnloadProgram()` при hot reload.
- Программа с регистрами сверх 128 отклоняется при загрузке.
- `CallNative`, чьи аргументы выходят за регистровый файл, отклоняется.
- Field binding, размер которого не соответствует opcode, отклоняется.
- Отсутствующий компонент даёт fault до записи.
- Отсутствующий command buffer даёт fault в `DespawnSelf`/`SpawnPrefab`.

### Отладка

Debug-сборка несёт необязательную секцию отладки после таблиц программы: один `DebugSpan { pc, line, column, flags }` на инструкцию, именованные локальные регистры и имя источника. Секция помечена в `ProgramHeader::flags`, поэтому

программа без неё загружается без изменений, а старые читатели игнорируют хвостовые байты. `TextScript` выдаёт span на инструкцию с 1-based строкой и колонкой и `DebugVariable` на каждый `let`; компилятор LogicGraph выдаёт один span узла графа на диапазон порождённых узлов (`DebugSpanGraphNode`). `ScriptVm::RunDebug()` исполняет тот же интерпретатор с принадлежащим вызывающему `VmDebugState`: breakpoints по pc (`SetBreakpoint`, битовая карта 8 KiB), бюджет одиночного шага, запрос паузы и продолжение, сохраняющее регистровый файл и pc. `VmStopInfo` сообщает `Completed`, `Faulted` или `Paused`, причину и pc. Хелперы `ScriptDebug` отображают pc обратно в позицию источника, перечисляют поля программы (разрешённые по схеме сцены) и читают или пишут живые значения полей.

Панель Script Debugger в редакторе компилирует текстовый скрипт, показывает источник с кликабельными breakpoints, Continue/Step/Pause/Reset, таблицу Locals, редактируемые поля Self, одношаговый стек и счётчики программы. `ScriptDebuggerPanelTest` покрывает ошибки компиляции, breakpoint по строке, продолжение и сообщение об ошибке без кадра ImGui.

### Дорожная карта

Следующие возможности языка, в порядке зависимостей:

12. Массивы, строки, map и пользовательские функции с фиксированным call stack.
13. Корутины/таймеры и детерминированное планирование.
14. Представления отладчика для канваса StateTree (breakpoints VM, spans и представление «Debug graph» для LogicGraph уже существуют).

Плоский контракт VM сознательно не расширяется heap-объектами в 8-байтовой инструкции. Любая новая объектная модель требует новой версии байткода и миграции.